

Operating Systems Lab

FINAL REPORT SUBMISSION

Lab Task no.	Experiment Name	Submission Status	Submission Date	Page
1	Introduction to Linux & Basic Commands	Yes ✓	7 th March, 2026	2-7
2	Linux Commands Exercise (Last 10 problems)	Yes ✓	5 th April, 2026	8-13
3	Shell Scripting Exercise Problems (Day-1)	Yes ✓	10 th April, 2026	14-19
4	Shell Scripting Exercise Problems (Day-2)	Yes ✓	17 th April, 2026	20-24
5	Implement SJF (preemptive) using C/Java or any other language.	Yes ✓	26 th April, 2026	25-27
6	Implement round robin scheduling algorithm using C/Java.	Yes ✓	2 nd May, 2026	28-30
7	Dining Philosopher Problem using Multithreading for both deadlock and no-deadlock conditions.	Yes ✓	21 st May, 2026	29-34

Submitted by,
Atik Jawad

ID: B220305043

Lab 1:

Introduction to Linux & Basic Commands

- Study of Linux OS structure
- File and directory commands: ls, pwd, cd, mkdir, rm, cp, mv
- File permissions: chmod, chown

File and Directory Commands

1. **ls** (List Directory Contents):

The *ls* command displays the files and directories present in the current working directory. It helps the user see what items exist inside a folder.

```
atik@Ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
```

2. **pwd** (Print Working Directory):

The *pwd* command displays the full path of the current directory where the user is working. It helps users know their exact location in the Linux file system hierarchy.

```
atik@Ubuntu:~$ pwd
/home/atik
```

3. **cd** (Change Directory):

The *cd* command is used to navigate between directories in the file system. It changes the current working directory to the specified folder.

```
atik@Ubuntu:~$ pwd
/home/atik
atik@Ubuntu:~$ cd Documents
atik@Ubuntu:~/Documents$ pwd
/home/atik/Documents
atik@Ubuntu:~/Documents$
```

Here we can see that by writing *cd* command in the terminal, we have changed the working directory from home to Documents.

4. **mkdir** (Make Directory):

The *mkdir* command is used to create a new directory (folder) in the current location. It helps organize files by creating separate directories.

Before:

```
atik@Ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
```

After:

```
atik@Ubuntu:~$ mkdir OSlab1
atik@Ubuntu:~$ ls
Desktop  Documents  Downloads  Music  OSlab1  Pictures  Public  snap  Templates  Videos
atik@Ubuntu:~$
```

5. **rm** (Remove Files or Directories):

The *rm* command is used to delete files or directories from the system. It's permanent and leaves no files or directories in the trash.

Before:

```
atik@Ubuntu:~/Desktop$ touch atik.txt
atik@Ubuntu:~/Desktop$ touch habijabi.txt
atik@Ubuntu:~/Desktop$ ls
atik.txt  habijabi.txt
```

We are now going to delete 'habijabi.txt' from desktop.

After:

```
atik@Ubuntu:~/Desktop$ rm habijabi.txt
atik@Ubuntu:~/Desktop$ ls
atik.txt
```

'habijabi.txt' is permanently deleted.

6. **cp** (Copy Files or Directories):

The *cp* command copies files or directories from one location to another. The original file remains unchanged while a duplicate is created.

Before:

```
atik@Ubuntu:~/Desktop$ ls
atik.txt
```

After: We will copy the 'atik.txt' file with the name of 'NotAtik.txt'.

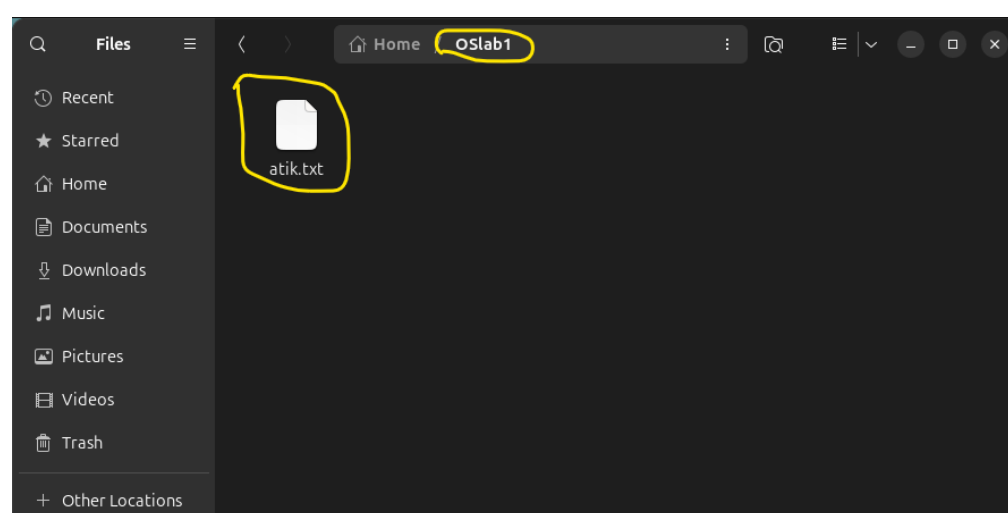
```
atik@Ubuntu:~/Desktop$ cp atik.txt NotAtik.txt
atik@Ubuntu:~/Desktop$ ls
atik.txt  NotAtik.txt
```

The 'atik.txt' is copied using *cp* command and pasted it in the same directory named 'NotAtik.txt'.

We can also copy files to different directories too. Using the following commands,

```
atik@Ubuntu:~/Desktop$ cp atik.txt /home/atik/OSlab1
atik@Ubuntu:~/Desktop$ cd /home/atik/OSlab1
atik@Ubuntu:~/OSlab1$ ls
atik.txt
```

We can verify by opening the OSlab1 directory.



7. **mv** (Move or Rename Files):

The *mv* command is used to move files from one directory to another or rename files. Unlike *cp*, it does not create a duplicate; it simply relocates or renames the file.

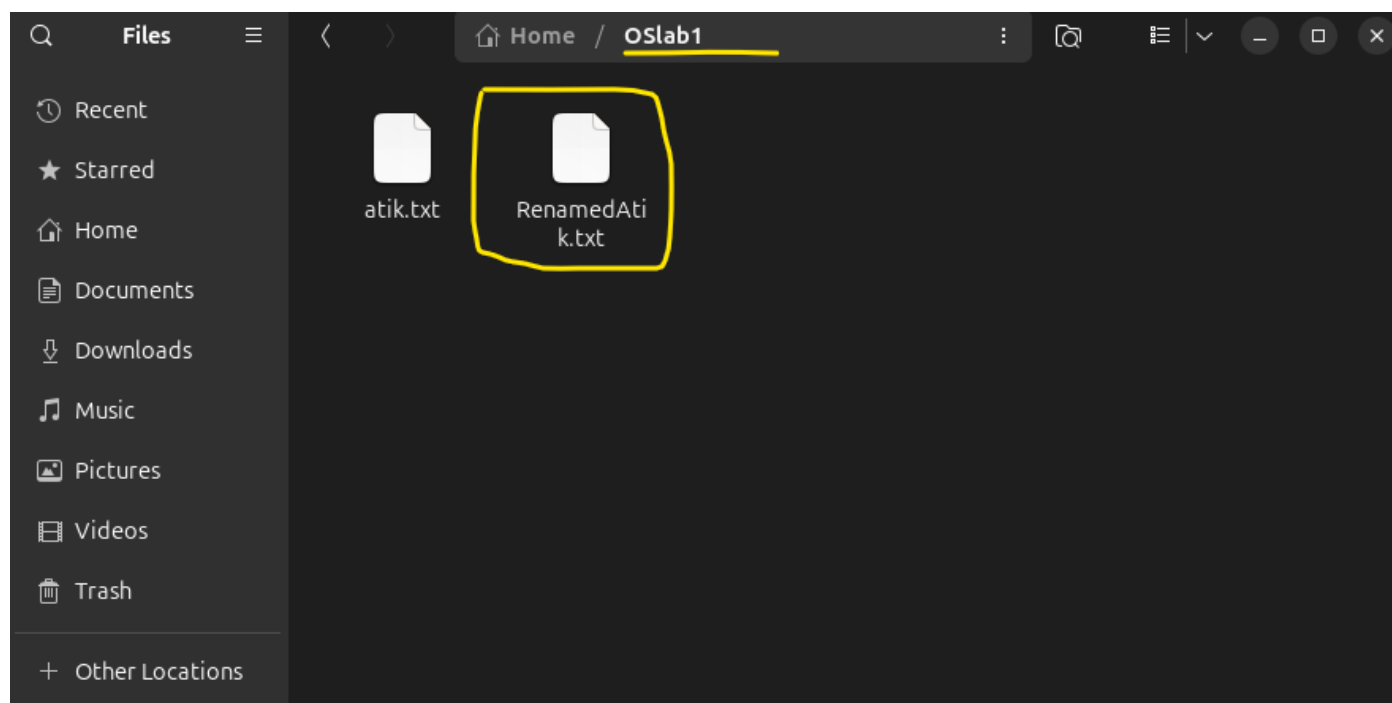
RENAMING: Renaming 'NotAtik.txt' to 'RenamedAtik.txt'

```
atik@Ubuntu:~/Desktop$ ls
atik.txt  NotAtik.txt
atik@Ubuntu:~/Desktop$ mv NotAtik.txt RenamedAtik.txt
atik@Ubuntu:~/Desktop$ ls
atik.txt  RenamedAtik.txt
```

MOVING: We moved the 'RenamedAtik.txt' from Desktop to OSlab1 directory.

```
atik@Ubuntu:~/Desktop$ mv RenamedAtik.txt /home/atik/OSlab1
atik@Ubuntu:~/Desktop$ cd /home/atik/OSlab1
atik@Ubuntu:~/OSlab1$ ls
atik.txt  RenamedAtik.txt
```

For more verification , we visited the directory.



File Permission Commands

8. **chmod** (Change File Permissions):

The *chmod* command changes the permission settings of a file or directory. Permissions determine who can read (r), write (w), or execute (x) the file.

We can see any files or directories permissions using *ls -l* command.

Before:

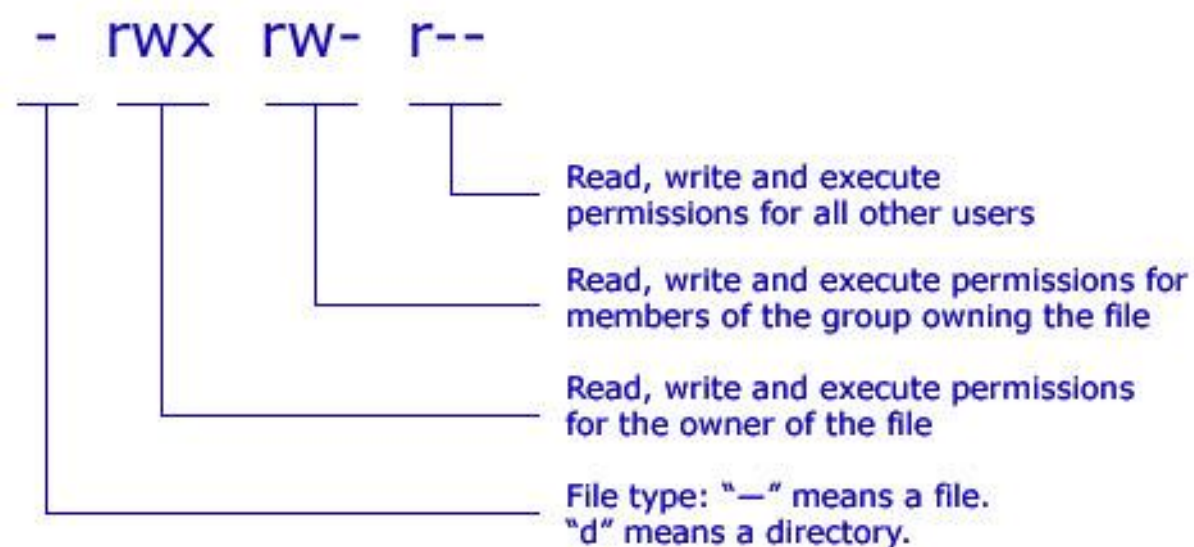
```

atik@Ubuntu:~$ ls -l
total 40
-rw-rw-r-- 1 atik atik  0 Mar  7 09:12 demo.txt
drwxr-xr-x 2 atik atik 4096 Mar  7 09:03 Desktop
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Documents
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Downloads
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Music
drwxrwxr-x 2 atik atik 4096 Mar  7 09:03 OSlab1
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Pictures
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Public
drwx----- 6 atik atik 4096 Mar  7 09:00 snap
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Templates
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Videos

```

The writing we can see in the image is the permission of the files and the directories in the **Home** directory.

Breaking down the permission,



After:

We will change the permission of the file using `chmod` command.

For example , we want ‘demo.txt’ **all** user to have the permission to “execute(x)”.

```

atik@Ubuntu:~$ chmod +x demo.txt
atik@Ubuntu:~$ ls -l
total 40
-rwxrwxr-x 1 atik atik  0 Mar  7 09:12 demo.txt
drwxr-xr-x 2 atik atik 4096 Mar  7 09:03 Desktop
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Documents
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Downloads
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Music
drwxrwxr-x 2 atik atik 4096 Mar  7 09:03 OSlab1
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Pictures
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Public
drwx----- 6 atik atik 4096 Mar  7 09:00 snap
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Templates
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Videos

```

We can also remove all the given permission.


```

atik@Ubuntu:~$ chmod -x demo.txt
atik@Ubuntu:~$ ls -l
total 40
-rw-rw-r-- 1 atik atik  0 Mar  7 09:12 demo.txt
drwxr-xr-x 2 atik atik 4096 Mar  7 09:03 Desktop
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Documents
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Downloads
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Music

```

9. **chown** (Change File Owner):

The *chown* command changes the ownership of a file or directory. It is usually used by system administrators to assign files to different users or groups.

Before:

```

atik@Ubuntu:~$ ls -l
total 40
-rw-rw-r-- 1 atik atik  0 Mar  7 09:12 demo.txt
drwxr-xr-x 2 atik atik 4096 Mar  7 09:03 Desktop
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Documents
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Downloads
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Music
drwxrwxr-x 2 atik atik 4096 Mar  7 09:03 OSlab1
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Pictures
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Public
drwx----- 6 atik atik 4096 Mar  7 09:00 snap
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Templates
drwxr-xr-x 2 atik atik 4096 Mar  7 06:45 Videos

```

We see both the owner here is 'atik', we can change it to something else using **chown**.

After:

```

atik@Ubuntu:~$ sudo chown root demo.txt
[sudo] password for atik:
atik@Ubuntu:~$ ls -l demo.txt
-rw-rw-r-- 1 root atik 0 Mar  7 09:12 demo.txt

```

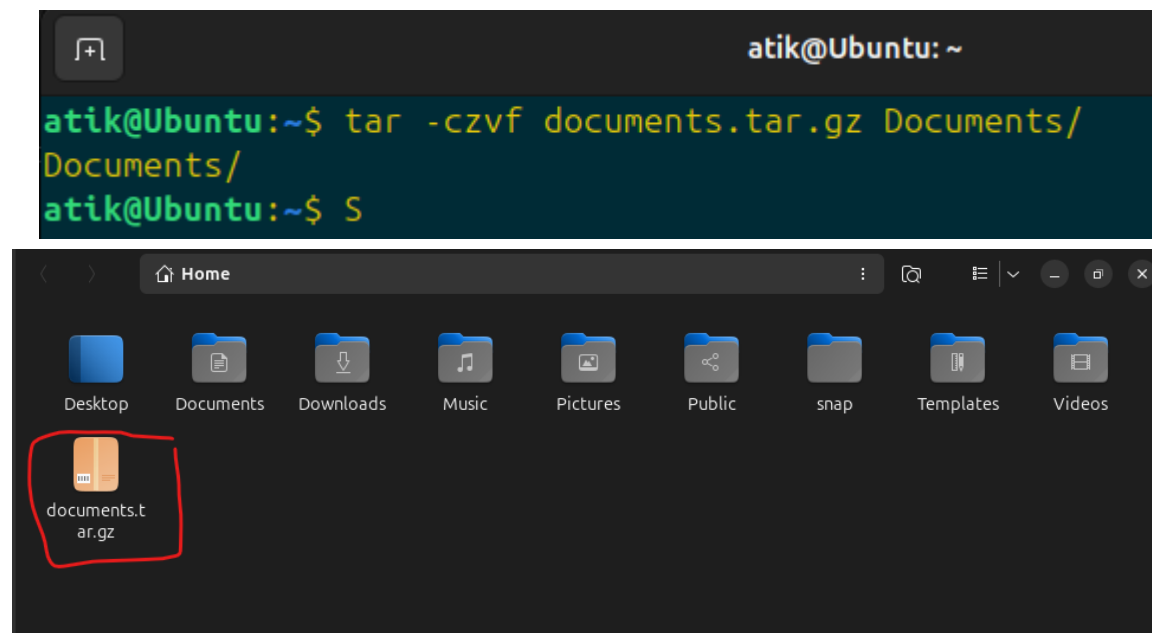
We can see the owner has been changed to 'root' after using **chown**. Note that, we have to type in '**sudo**' before when using **chown**.

Lab 2:

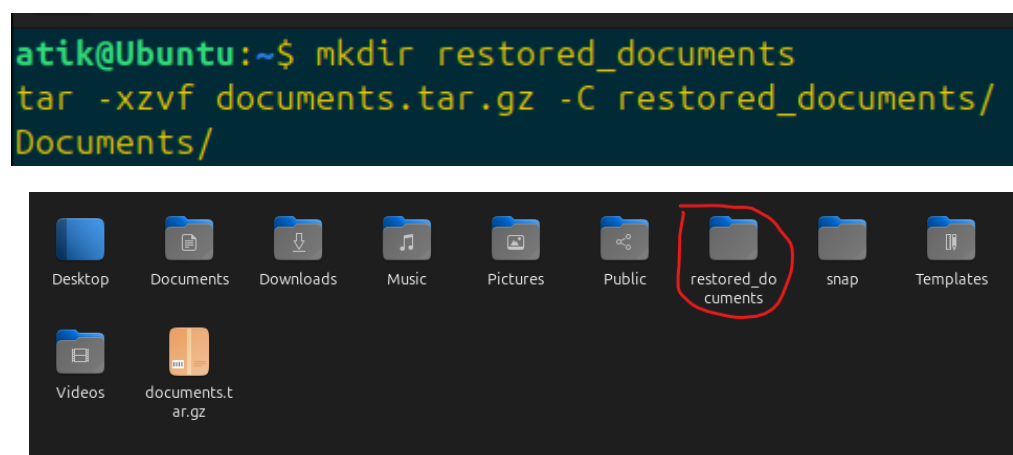
Solve the following Problems with Linux Commands

1. Create a compressed archive of the "documents" directory named "documents.tar.gz" using the tar and gzip commands.
2. Extract the contents of the compressed archive into a new directory called "restored_documents".
3. Rename the extracted directory to "backup" using the mv command.
4. Use the ls command to list the contents of the "backup" directory.
5. Find all files in the current directory with a ".txt" extension.
6. Search for a specific string within all text files in a directory.
7. Count the number of lines in a text file.
8. Sort the contents of a text file alphabetically.
9. Display the last 10 lines of a text file.
10. Find and replace a specific word in a text file.
11. View the contents of a file while continuously updating it as new lines are added (tail command).
12. Display detailed information about a file or directory (ls -l).
13. Check the amount of free disk space on your system (df -h).
14. Monitor real-time network traffic (iftop).
15. Set up a new environment variable and export it.
16. Create a symbolic link to a file or directory.
17. Check the status of a running service (systemctl status).
18. View the contents of a compressed file without decompressing it (zcat or zless).
19. Use grep to search for a pattern in a file or multiple files.
20. Find all files modified within the last 24 hours in a specific directory.

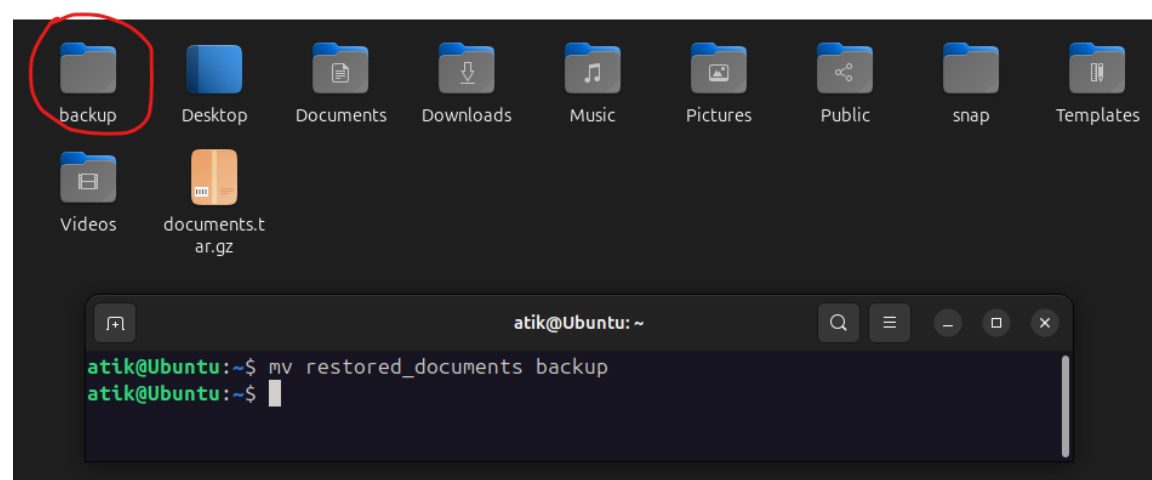
1. Create a compressed archive of the "documents" directory named "documents.tar.gz" using tar and gzip commands.



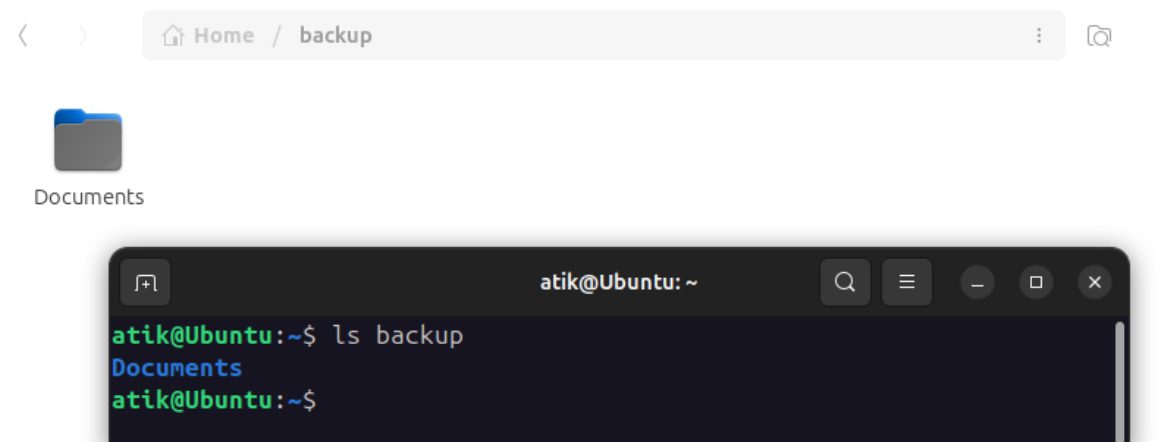
2. Extract the contents of the compressed archive into a new directory called "restored_documents".



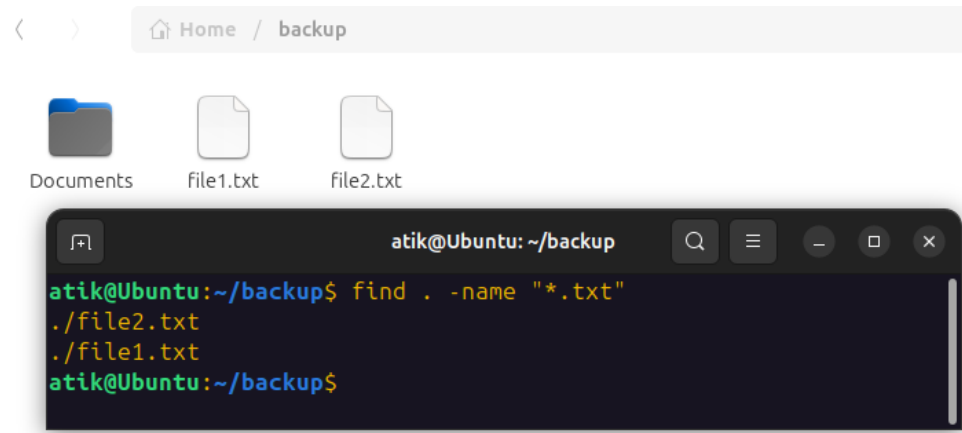
3. Rename the extracted directory to "backup".



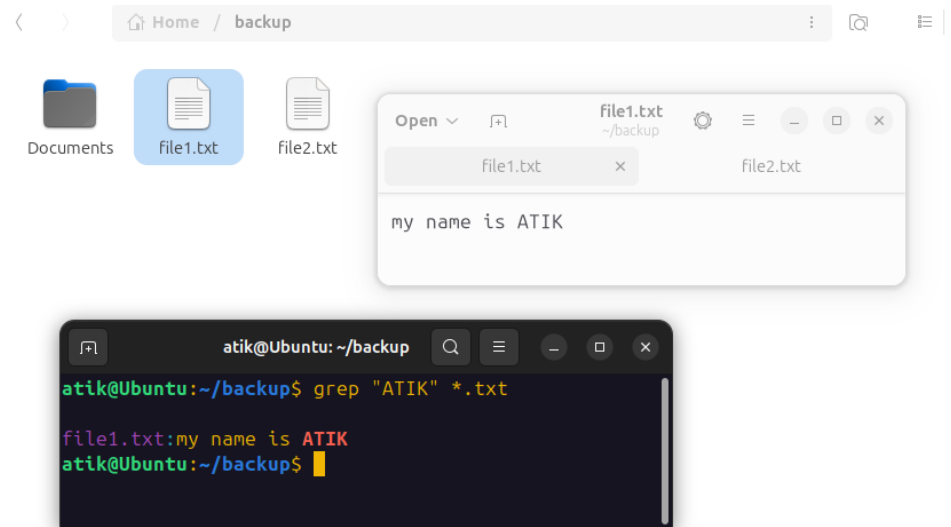
4. List the contents of the "backup" directory.



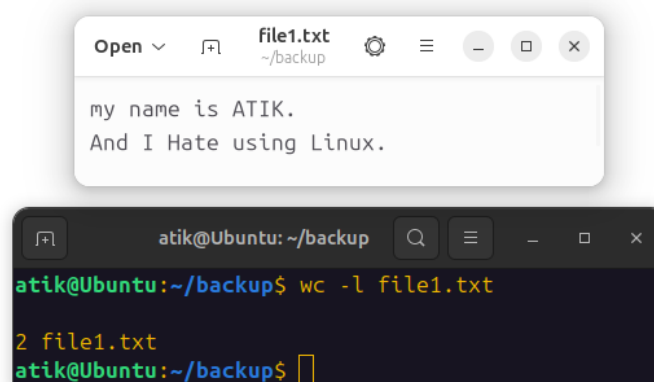
5. Find all files with a ".txt" extension.



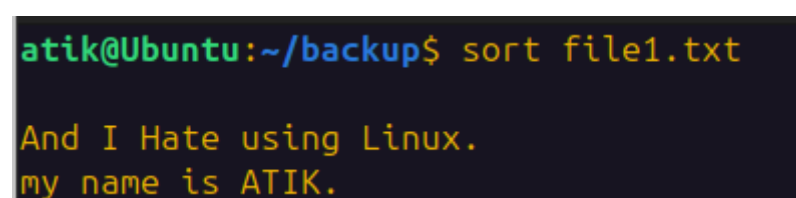
6. Search for a specific string within all text files.



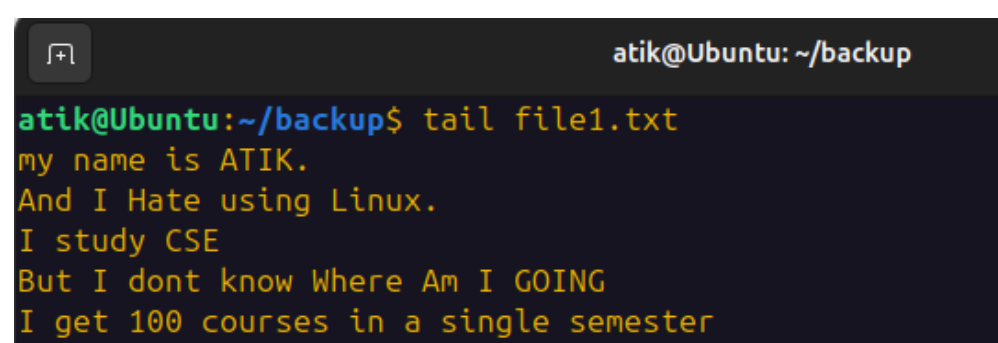
7. Count the number of lines in a text file.



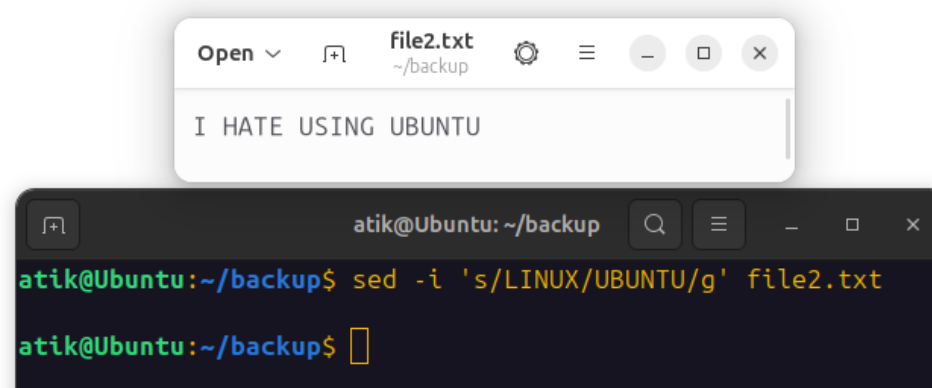
8. Sort contents of a text file alphabetically.



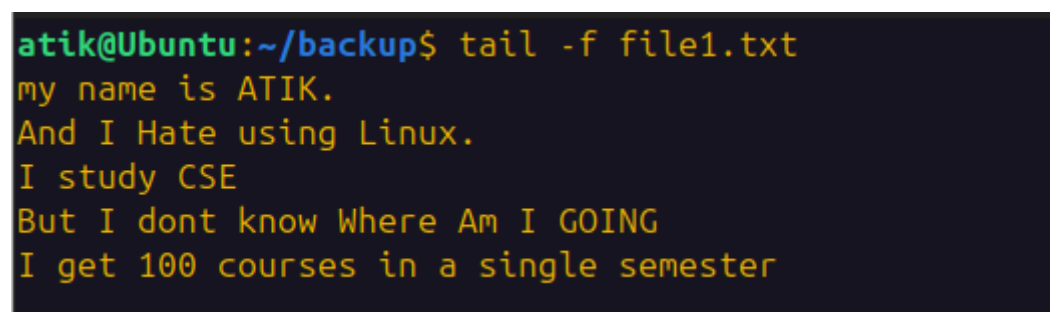
9. Display the last 10 lines of a text file.



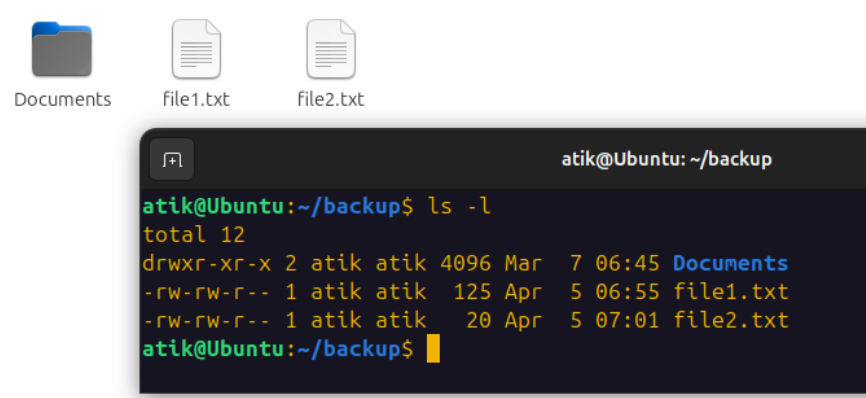
10. Find and replace a word in a text file.



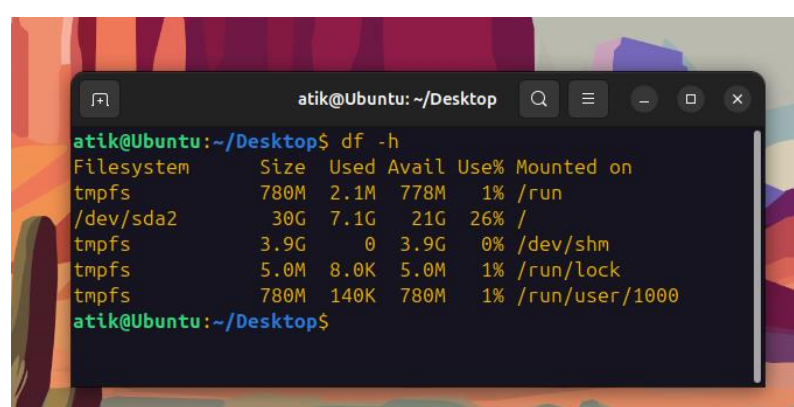
11. View file updates continuously.



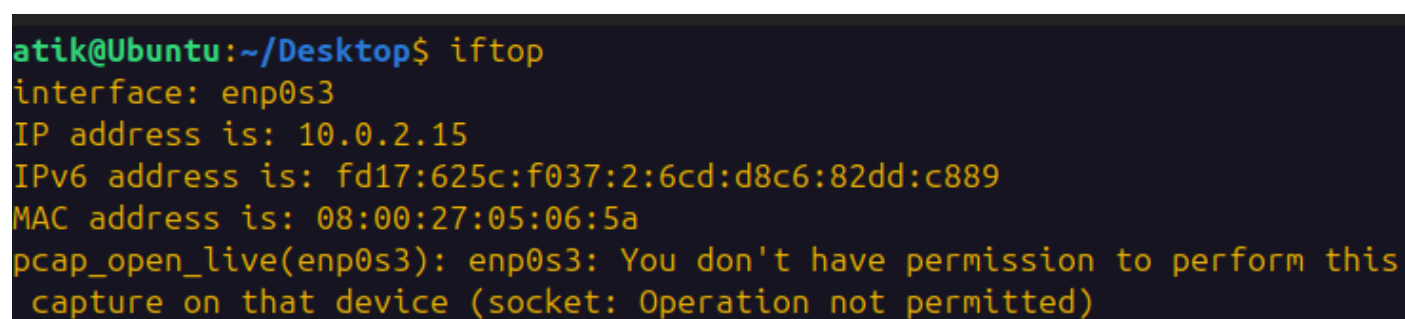
12. Display detailed file information.



13. Check the amount of free disk space.



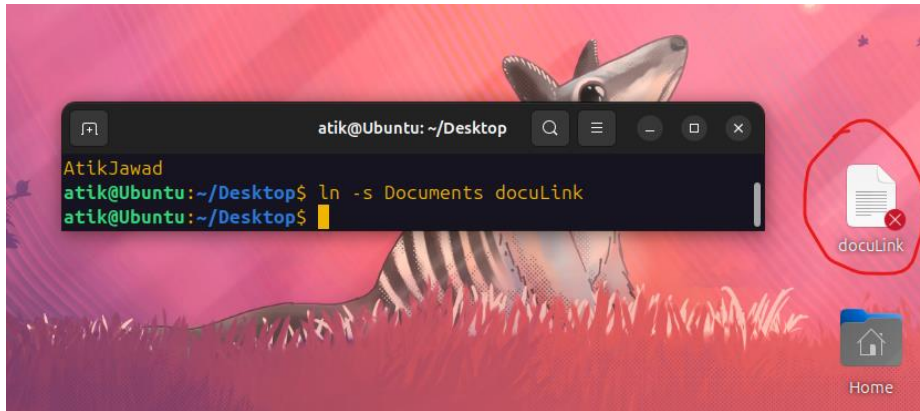
14. Monitor real-time network traffic. (iftop)



15. Set and export an environment variable.

```
atik@Ubuntu:~/Desktop$ export MY_VAR="AtikJawad"
echo $MY_VAR
AtikJawad
```

16. Create a symbolic link.



17. Check status of a running service.

```
atik@Ubuntu:~/Desktop$ systemctl status bluetooth
Warning: The unit file, source configuration file or drop-ins of bluetooth.serv>
○ bluetooth.service - Bluetooth service
   Loaded: loaded (/usr/lib/systemd/system/bluetooth.service; enabled; preset>
   Active: inactive (dead)
     Docs: man:bluetoothd(8)
lines 1-5/5 (END)
```

18. View contents of a compressed file without extracting.

```
atik@Ubuntu:~$ zcat documents.tar.gz

Documents/0000755000175000017500000000000015152744613011471 5ustar  atikati
atik@Ubuntu:~$
```

19. Use *grep* to search for a pattern in a file or multiple files.

Single file:

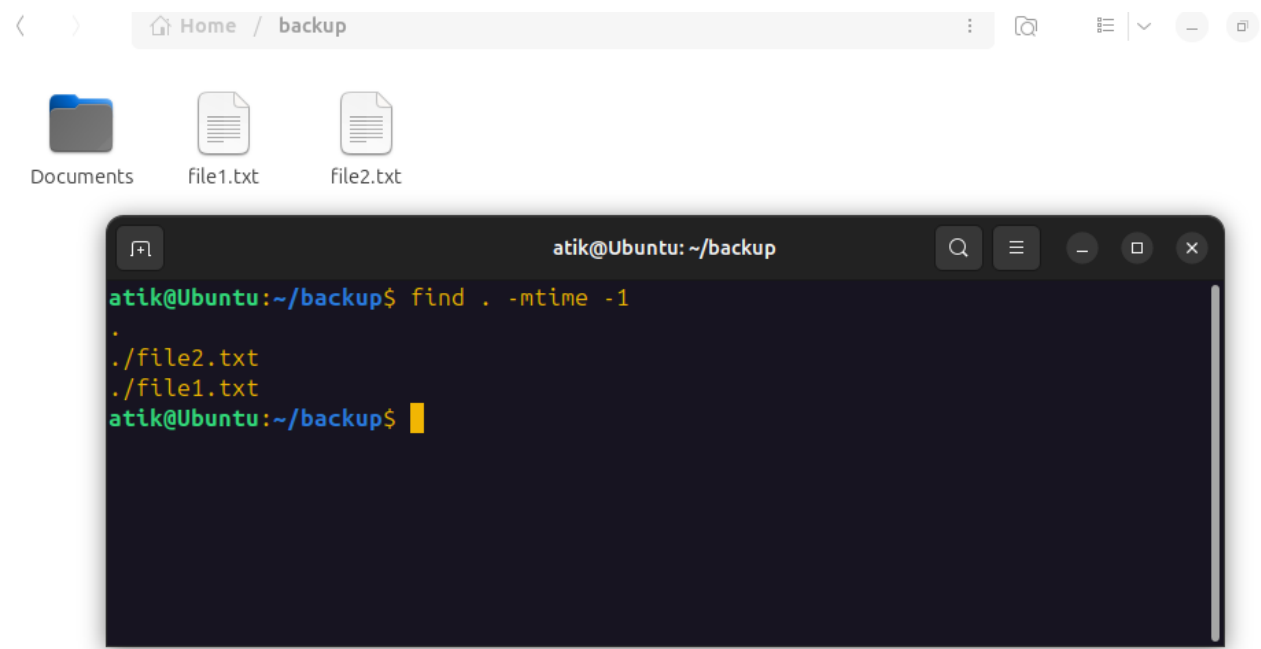
```
atik@Ubuntu:~/backup$ grep "ATIK" file1.txt
my name is ATIK.
This is ATIK jawad speaking to the WALL.
```

Multiple file:

```
atik@Ubuntu:~/backup$ grep "linux" *.txt

file1.txt:And I Hate using linux.
file2.txt:linux was never my choiceee
atik@Ubuntu:~/backup$
```

20. Find files modified in the last 24 hours.



Lab 3:

Shell Scripting Exercise Problems (Day-1)

- | |
|--|
| 1. Write a script to display the current date and time. |
| 2. Create a script that takes a filename as an argument and displays its contents. |
| 3. Write a script to check if a directory exists, and if not, create it. |
| 4. Create a script that counts the number of files in a directory. |
| 5. Write a backup script that compresses a specified directory and stores it in a backup folder. |

1. Write a script to display the current date and time.

Shell Script:

```
#!/bin/bash
# Description: Displays current date and time

echo "Current Date and Time:"
date
```

Terminal Output:

```
atik@Ubuntu:~$ ls
Desktop    Downloads  Music      Public    Templates
Documents  lab3.sh    Pictures   snap      Videos
atik@Ubuntu:~$ chmod +x lab3.sh
atik@Ubuntu:~$ ./lab3.sh
Current Date and Time:
Fri Apr 10 09:19:42 AM UTC 2026
```


2. Create a script that takes a filename as an argument and displays its contents.

Shell Script:

```
#!/bin/bash
# Description: Displays contents of a file

if [ -f "$1" ];
then
    echo "Contents of file:"
    cat "$1"
else
    echo "File does not exist!"
fi
```

Terminal Output:

```
atik@Ubuntu:~$ ls
Desktop  Documents  Downloads  file1.txt  lab3.sh  Music  Pictures
Public  snap  Templates  Videos
atik@Ubuntu:~$ ./lab3.sh test.txt

File does not exist!
atik@Ubuntu:~$ ^C
atik@Ubuntu:~$ ./lab3.sh file1.txt

Contents of file:
My name is ATIK JAWAD
```

3. Write a script to check if a directory exists, and if not, create it.

Shell Script:

```
#!/bin/bash
# Description: Checks directory and creates if not exists

if [ -d "$1" ]; then
    echo "Directory already exists."
else
    mkdir "$1"
    echo "Directory created."
fi
```

Terminal Output:

```
atik@Ubuntu:~$ ls
Desktop  Downloads  lab3.sh  Pictures  snap      Videos
Documents  file1.txt  Music    Public    Templates

atik@Ubuntu:~$ ./lab3.sh Music
Directory already exists.
atik@Ubuntu:~$ ./lab3.sh OSLab
Directory created.
atik@Ubuntu:~$ ls
Desktop  Downloads  lab3.sh  OSLab    Public  Templates
Documents  file1.txt  Music    Pictures  snap    Videos
```

4. Create a script that counts the number of files in a directory.

Shell Script:

```
#!/bin/bash
# Description: Count files in a directory

echo "Enter directory name:"
read folder

if [ -d "$folder" ];
then
    count=$(ls "$folder" | wc -l) # | (pipe operator) : Take output from left → send it to right
                                # $(...) : This runs a command and gives its result
    echo "Number of files: $count"
else
    echo "Directory does not exist!"
fi
```

Terminal Output:

```
atik@Ubuntu:~$ ./lab3.sh
Enter directory name:
Documents
Number of files: 2
atik@Ubuntu:~$ cd Documents
atik@Ubuntu:~/Documents$ ls
file1.txt file2.txt
atik@Ubuntu:~/Documents$
```

5. Write a backup script that compresses a specified directory and stores it in a backup folder.

Shell Script:

```
#!/bin/bash
# Description: Creates backup of a directory

source_dir=$1 # 1st argument
backup_dir=$2 # 2nd argument

if [ ! -d "$source_dir" ];
then
    echo "Source directory does not exist!"
    exit 1
fi

mkdir -p "$backup_dir" # -p = Don't complaint if it already exists

filename="backup_$(date).tar.gz" #this creates a name like: backup_20260410.tar.gz

tar -czvf "$backup_dir/$filename" "$source_dir"

echo "Backup completed: $backup_dir/$filename"
```

Terminal Output:

```
atik@Ubuntu:~$ ls
Backups  Documents  lab3.sh  OSLab    Public  Templates
Desktop  Downloads  Music    Pictures  snap    Videos
atik@Ubuntu:~$ ./lab3.sh OSLab Backups
OSLab/
Backup completed: Backups/backup_Fri Apr 10 10:59:10 AM UTC 2026.tar.gz
atik@Ubuntu:~$ ./lab3.sh NonExistingFolder Backups
Source directory does not exist!
atik@Ubuntu:~$
```

Lab 4:

Shell Scripting Exercise Problems (Day-2)

Write shell scripting programs for the following list of problems.

1. You want to collect information about the system, such as CPU usage, memory usage, disk space, and network statistics.
2. You want to generate random passwords of a specified length.
3. You need to monitor system resources continuously and send an email notification (if possible, otherwise just print message) if certain thresholds (e.g., high CPU usage, low disk space) are exceeded.
4. In a multi-user computer system (such as in offices, universities, or shared servers), multiple users access the system at different times. However, there is no proper tracking mechanism to monitor who accessed the system, when they logged in, and from where. Develop a shell scripting solution that automatically creates and maintains a log file recording user access history, including login time and user details.
5. Unauthorized users may try to access the system by guessing passwords. Without monitoring, these attempts go unnoticed. Develop a script that records and reports failed login attempts.
6. Some applications consume excessive CPU or memory, slowing down the system, but users are unaware of which process is responsible. Create a script to monitor running processes and alert when resource usage is too high.
7. Administrators may need to track how long a system has been running without rebooting. Create a script that logs system uptime regularly.

1. Write a script to collect information about the system, such as CPU usage, memory usage, disk space, and network statistics.

Shell Script:

```
#!/bin/bash
# Description: Shows system info

echo "CPU Usage:"
top -bn1 | grep "Cpu"

echo "Memory Usage:"
free -h

echo "Disk Usage:"
df -h

echo "Network Info:"
ifconfig|
```

Terminal Output:

```
atik@Ubuntu:~$ ./lab4.sh
CPU Usage:
%Cpu(s):  6.8 us,  2.3 sy,  0.0 ni, 90.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
Memory Usage:
Mem:          total      used      free      shared  buff/cache   available
Swap:         0B         0B         0B
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           583M  1.5M  582M   1% /run
/dev/sda2       25G   5.7G   18G  25% /
tmpfs           2.9G   0     2.9G   0% /dev/shm
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
tmpfs           583M  124K  583M   1% /run/user/1000
Network Info:
./lab4.sh: line 14: ifconfig: command not found
```

2. Create a script that generates random passwords of a specified length.

Shell Script:

```
#!/bin/bash
# Description: Generate random password

echo "Enter password length:"
read length

tr -dc A-Za-z0-9 </dev/urandom | head -c $length
echo
```

Terminal Output:

```
atik@Ubuntu:~$ ./lab4.sh
Enter password length:
8
YN96isDg
```


3. Write a script to monitor system resources and print a message if certain thresholds (e.g., high CPU usage, low disk space) are exceeded.

Shell Script:

```
#!/bin/bash

echo "CPU usage check:"
top -n 1|
```

Terminal Output:

Tasks: 235 total, 1 running, 234 sleeping, 0 stopped, 0 zombie

%Cpu(s): 9.1 us, 4.5 sy, 0.0 ni, 86.4 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st

MiB Mem : 5828.1 total, 1052.4 free, 2481.0 used, 2416.3 buff/cache

MiB Swap: 0.0 total, 0.0 free, 0.0 used. 3347.1 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4216	atik	20	0	23192	5752	3540	R	9.1	0.1	0:00.02	top
1	root	20	0	23120	14476	9812	S	0.0	0.2	0:01.80	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.01	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
10	root	20	0	0	0	0	I	0.0	0.0	0:00.01	kworker/0:1-cgroup_bpf_destroy
11	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
12	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/u16:0-ipv6_addrconf
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
14	root	20	0	0	0	0	S	0.0	0.0	0:00.03	ksoftirqd/0
15	root	20	0	0	0	0	I	0.0	0.0	0:00.80	rcu_preempt
16	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_worker/0
17	root	20	0	0	0	0	S	0.0	0.0	0:00.02	rcu_exp_gp_kthread_worker
18	root	rt	0	0	0	0	S	0.0	0.0	0:00.01	migration/0
19	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/0
20	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
21	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
22	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/1
23	root	rt	0	0	0	0	S	0.0	0.0	0:00.42	migration/1
24	root	20	0	0	0	0	S	0.0	0.0	0:00.03	ksoftirqd/1
26	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/1:0H-events_highpri

atik@Ubuntu:~\$

4. Develop a shell scripting solution that automatically creates and maintains a log file recording user access history, including login time and user details.

Shell Script:

```
#!/bin/bash
# Description: Log user access

echo "User Log History:" >> userlog.txt
who >> userlog.txt
echo "-----" >> userlog.txt|
```


Output:

```
User Log History:
atik      seat0      2026-04-17 16:33 (login screen)
atik      tty2       2026-04-17 16:33 (tty2)
-----
User Log History:
atik      seat0      2026-04-17 16:33 (login screen)
atik      tty2       2026-04-17 16:33 (tty2)
-----
User Log History:
atik      seat0      2026-04-17 16:33 (login screen)
atik      tty2       2026-04-17 16:33 (tty2)
-----
```

5. Develop a script that records and reports failed login attempts.

Shell Script:

```
#!/bin/bash
# Description: Show failed login attempts

echo "Failed Login Attempts:"
grep "Failed password" /var/log/auth.log
```

Terminal Output:

```
atik@Ubuntu:~$ sudo ./lab4.sh
[sudo] password for atik:
Sorry, try again.
[sudo] password for atik:
Sorry, try again.
[sudo] password for atik:
Failed Login Attempts:
atik@Ubuntu:~$ ./lab4.sh
Failed Login Attempts:
grep: /var/log/auth.log: Permission denied
atik@Ubuntu:~$
```

6. Create a script to monitor running processes and alert when resource usage is too high.

Shell Script:

```
#!/bin/bash
# Description: Show high CPU processes

echo "Top CPU consuming processes:"
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head
```

Terminal Output:

```
atik@Ubuntu:~$ ./lab4.sh
Top CPU consuming processes:
  PID      PPID  CMD                                %MEM %CPU
  4771      4770  ps -eo pid,ppid,cmd,%mem,%c      0.0  100
  1916      1662  /usr/bin/gnome-shell              11.0  13.0
  2732      1916  /snap/firefox/7766/usr/lib/       9.4   5.3
  2707      1662  /usr/bin/gnome-text-editor        5.5   5.2
  3290      2888  /snap/firefox/7766/usr/lib/      11.4   4.0
  4768         2  [kworker/u18:0-events_unbou       0.0   1.5
  4466      1662  /usr/bin/nautilus --gapplic        5.4   0.7
  4461         2  [kworker/u18:2-events_unbou       0.0   0.5
   177         2  [kworker/u19:2-events_power       0.0   0.5
atik@Ubuntu:~$
```

7. Create a script that logs system uptime regularly.

Shell Script:

```
#!/bin/bash
# Description: Log system uptime

echo "System Uptime:" >> uptime.txt
uptime >> uptime.txt
echo "------" >> uptime.txt
```

Output:

```
System Uptime:
17:13:02 up 40 min,  1 user,  load average: 0.07, 0.20, 0.25
-----
System Uptime:
17:13:07 up 40 min,  1 user,  load average: 0.07, 0.20, 0.25
-----
System Uptime:
17:13:11 up 40 min,  1 user,  load average: 0.06, 0.19, 0.25
-----
```

Lab 5:

Implement SJF (preemptive) using C/Java or any other language. You should use different types of test cases in the input/output system.

Python Code:

```
n = int(input("Enter number of processes: "))

p = ["P"+str(i+1) for i in range(n)]

at = list(map(int, input(f"Enter Arrival time for {n} processes serially :").split()))
bt = list(map(int, input(f"Enter Burst time for {n} processes serially :").split()))

rem = bt[:]
wt = [0]*n

time = 0
done = 0
prev = -1

print("\nGantt Chart:")
print("0", end="")

while done < n:
    idx = -1
    for i in range(n):
        if at[i] <= time and rem[i] > 0:
            if idx == -1 or rem[i] < rem[idx]:
                idx = i

    if idx == -1:
        time += 1
        continue

    if prev != idx:
        print(" ->", p[idx], end="")
        prev = idx

    rem[idx] -= 1
    time += 1

    if rem[idx] == 0:
        done += 1
        wt[idx] = time - bt[idx] - at[idx]
```

```

print(" ->", time)

print("\nProcess\t\tArrival\t\tBurst\t\tWaiting")

for i in range(n):

print(p[i], "\t\t", at[i] , "\t\t", bt[i], "\t\t", wt[i])

print("\nAvg Waiting Time = ", sum(wt)/n)

```

Taste Run 1:

```

PS C:\Users\atikj> & C:/Users/atikj/AppData/Local/Python/pythoncore-3

Enter number of processes: 4
Enter Arrival time for 4 processes serially : 0 1 2 3
Enter Burst time for 4 processes serially : 6 8 7 3

Gantt Chart:
0 -> P1 -> P4 -> P3 -> P2 -> 24

Process      Arrival      Burst      Waiting
P1           0           6           0
P2           1           8          15
P3           2           7           7
P4           3           3           3

Avg Waiting Time = 6.25
PS C:\Users\atikj> 

```

Taste Run 2:

```

PS C:\Users\atikj> & C:/Users/atikj/AppData/Local/Python/pythoncore-3

Enter number of processes: 3
Enter Arrival time for 3 processes serially : 0 5 6
Enter Burst time for 3 processes serially : 4 3 2

Gantt Chart:
0 -> P1 -> P2 -> P3 -> 10

Process      Arrival      Burst      Waiting
P1           0           4           0
P2           5           3           0
P3           6           2           2

Avg Waiting Time = 0.6666666666666666
PS C:\Users\atikj> 

```

Lab 6:

Implement round robin scheduling algorithm using C/Java/Python. Print the results in the following tabular format:

Process	BurstTime	startTime	endTime	WaitingTime
P1	3	0	0	3
P2	4	3	3	7

Code:

```
n = int(input("Enter number of processes: "))

p = ["P"+str(i+1) for i in range(n)]
at = list(map(int, input("Enter AT: ").split()))
bt = list(map(int, input("Enter BT: ").split()))
q = int(input("Enter Time Quantum: "))

rem = bt[:]
time = 0

start = [-1]*n
end = [0]*n

queue = []
visited = [0]*n

while True:
    for i in range(n):
        if at[i] <= time and visited[i] == 0:
            queue.append(i)
            visited[i] = 1

    if len(queue) == 0:
        time += 1
        continue

    i = queue.pop(0)

    if start[i] == -1:
        start[i] = time

    if rem[i] > q:
        time += q
        rem[i] -= q
    else:
        time += rem[i]
        rem[i] = 0
        end[i] = time

    for j in range(n):
        if at[j] <= time and visited[j] == 0:
            queue.append(j)
            visited[j] = 1

    if rem[i] > 0:
        queue.append(i)

    if all(r == 0 for r in rem):
        break

# waiting time
wt = [0]*n
for i in range(n):
    wt[i] = end[i] - bt[i] - at[i]

# Output
print("\nProcess\tBT\tStart\tEnd\tWT")
for i in range(n):
    print(p[i], "\t", bt[i], "\t", start[i], "\t", end[i], "\t", wt[i])
```

Test Run 1:

```
PS C:\Users\Atik Jawad> & "C:/Users/Atik Jawad/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Atik Jawad/Desktop/lab.py"
Enter number of processes: 4
Enter AT: 2 3 1 6
Enter BT: 4 2 5 1
Enter Time Quantum: 2

Process BT      Start  End    WT
P1      4        3     11    5
P2      2        5      7     2
P3      5        1     13     7
P4      1       11     12     5
PS C:\Users\Atik Jawad> 
```

Test Run 2:

```
PS C:\Users\Atik Jawad> & "C:/Users/Atik Jawad/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Atik Jawad/Desktop/lab.py"
Enter number of processes: 5
Enter AT: 3 2 4 1 5
Enter BT: 6 7 3 2 5
Enter Time Quantum: 3

Process BT      Start  End    WT
P1      6        3     18     9
P2      7        6     24    15
P3      3        9     12     5
P4      2        1      3      0
P5      5       12     23    13
PS C:\Users\Atik Jawad> 
```

Test Run 3:

```
PS C:\Users\Atik Jawad> & "C:/Users/Atik Jawad/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Atik Jawad/Desktop/lab.py"
Enter number of processes: 7
Enter AT: 2 3 1 5 7 9 2
Enter BT: 3 1 2 9 7 4 2
Enter Time Quantum: 2

Process BT      Start  End    WT
P1      3        3     11     6
P2      1        5      6      2
P3      2        1      3      0
P4      9        8     29    15
P5      7       11     28    14
P6      4       13     21      8
P7      2        6      8      4
PS C:\Users\Atik Jawad> 
```


Lab 7:

Dining Philosopher Problem using Multithreading for both deadlock and no-deadlock conditions.

[Python code:](#)

```
# Dining Philosopher Problem using Multithreading
# -----
# This file shows TWO versions:
#
# 1. Deadlock Version
#   All philosophers pick LEFT fork first.
#   This can create circular waiting -> DEADLOCK.
#
# 2. No-Deadlock Version
#   One philosopher picks RIGHT fork first.
#   This breaks circular waiting -> NO DEADLOCK.

import threading
import time
import random

NUM_PHILOSOPHERS = 5

# DEADLOCK VERSION

print("\n===== DEADLOCK VERSION =====\n")

# Create forks (locks)
forks = [threading.Lock() for _ in range(NUM_PHILOSOPHERS)]

def philosopher_deadlock(i):
    left_fork = forks[i]
    right_fork = forks[(i + 1) % NUM_PHILOSOPHERS]

    while True:
        print(f"Philosopher {i} is THINKING")
        time.sleep(random.uniform(1, 2))

        print(f"Philosopher {i} wants LEFT fork")
        left_fork.acquire()
        print(f"Philosopher {i} picked LEFT fork")

        time.sleep(1) # increases deadlock chance

        print(f"Philosopher {i} wants RIGHT fork")
        right_fork.acquire()
        print(f"Philosopher {i} picked RIGHT fork")

        print(f"Philosopher {i} is EATING")
        time.sleep(random.uniform(1, 2))

        left_fork.release()
        right_fork.release()

        print(f"Philosopher {i} released both forks\n")

threads = []

for i in range(NUM_PHILOSOPHERS):
    t = threading.Thread(target=philosopher_deadlock, args=(i,))
    t.daemon = True
    threads.append(t)
    t.start()

# Run for some time
time.sleep(15)

print("\nPossible DEADLOCK occurred above ☹️\n")

# NO-DEADLOCK VERSION

print("\n===== NO-DEADLOCK VERSION =====\n")

# New set of forks
forks = [threading.Lock() for _ in range(NUM_PHILOSOPHERS)]
```

```

def philosopher_no_deadlock(i):
    left_fork = forks[i]
    right_fork = forks[(i + 1) % NUM_PHILOSOPHERS]

    while True:
        print(f"Philosopher {i} is THINKING")
        time.sleep(random.uniform(1, 2))

        # Last philosopher picks RIGHT fork first
        # This breaks circular wait condition
        if i == NUM_PHILOSOPHERS - 1:
            first_fork = right_fork
            second_fork = left_fork
        else:
            first_fork = left_fork
            second_fork = right_fork

        print(f"Philosopher {i} is waiting for forks")

        with first_fork:
            print(f"Philosopher {i} picked first fork")

            time.sleep(0.5)

            with second_fork:
                print(f"Philosopher {i} picked second fork")

                print(f"Philosopher {i} is EATING")
                time.sleep(random.uniform(1, 2))

                print(f"Philosopher {i} finished EATING\n")

threads = []

for i in range(NUM_PHILOSOPHERS):
    t = threading.Thread(target=philosopher_no_deadlock, args=(i,))
    t.daemon = True
    threads.append(t)
    t.start()

# Keep program alive
time.sleep(20)

print("\n No deadlock occurred.")

```

Terminal Output:

```

===== DEADLOCK VERSION =====

Philosopher 0 is THINKING
Philosopher 1 is THINKING
Philosopher 2 is THINKING
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 2 wants LEFT fork
Philosopher 2 picked LEFT fork
Philosopher 0 wants LEFT fork
Philosopher 0 picked LEFT fork
Philosopher 4 wants LEFT fork
Philosopher 4 picked LEFT fork
Philosopher 3 wants LEFT fork
Philosopher 3 picked LEFT fork
Philosopher 1 wants LEFT fork
Philosopher 1 picked LEFT fork
Philosopher 2 wants RIGHT fork
Philosopher 0 wants RIGHT fork
Philosopher 4 wants RIGHT fork
Philosopher 3 wants RIGHT fork
Philosopher 1 wants RIGHT fork

Possible DEADLOCK occurred above ☹️

```

===== NO-DEADLOCK VERSION =====

Philosopher 0 is THINKING	Philosopher 2 is THINKING	Philosopher 2 finished EATING	Philosopher 3 is waiting for forks
Philosopher 1 is THINKING	Philosopher 0 finished EATING		Philosopher 2 finished EATING
Philosopher 2 is THINKING		Philosopher 2 is THINKING	
Philosopher 3 is THINKING	Philosopher 0 is THINKING	Philosopher 3 is waiting for forks	Philosopher 2 is THINKING
Philosopher 4 is THINKING	Philosopher 1 picked first fork	Philosopher 3 picked first fork	Philosopher 1 picked second fork
Philosopher 2 is waiting for forks	Philosopher 4 picked first fork	Philosopher 1 picked second fork	 Philosopher 1 is EATING
Philosopher 2 picked first fork	Philosopher 3 is waiting for forks	 Philosopher 1 is EATING	Philosopher 3 picked first fork
Philosopher 0 is waiting for forks	Philosopher 3 picked first fork	Philosopher 4 picked second fork	Philosopher 0 is waiting for forks
Philosopher 0 picked first fork	Philosopher 1 picked second fork	 Philosopher 4 is EATING	Philosopher 4 finished EATING
Philosopher 2 picked second fork	 Philosopher 1 is EATING	Philosopher 2 is waiting for forks	
 Philosopher 2 is EATING	Philosopher 4 picked second fork	Philosopher 0 is waiting for forks	Philosopher 4 is THINKING
Philosopher 1 is waiting for forks	 Philosopher 4 is EATING	Philosopher 1 finished EATING	Philosopher 0 picked first fork
Philosopher 1 picked first fork	Philosopher 2 is waiting for forks		Philosopher 3 picked second fork
Philosopher 3 is waiting for forks	Philosopher 0 is waiting for forks	Philosopher 1 is THINKING	 Philosopher 3 is EATING
Philosopher 4 is waiting for forks	Philosopher 4 finished EATING	Philosopher 2 picked first fork	Philosopher 2 is waiting for forks
Philosopher 2 finished EATING		Philosopher 4 finished EATING	Philosopher 1 finished EATING
	Philosopher 4 is THINKING		
Philosopher 2 is THINKING	Philosopher 3 picked second fork	Philosopher 4 is THINKING	Philosopher 1 is THINKING
Philosopher 1 picked second fork	 Philosopher 3 is EATING	Philosopher 3 picked second fork	Philosopher 2 picked first fork
Philosopher 3 picked first fork	Philosopher 0 picked first fork	 Philosopher 3 is EATING	Philosopher 0 picked second fork
 Philosopher 1 is EATING	Philosopher 1 finished EATING	Philosopher 0 picked first fork	 Philosopher 0 is EATING
Philosopher 3 picked second fork		Philosopher 0 picked second fork	Philosopher 3 finished EATING
 Philosopher 3 is EATING	Philosopher 1 is THINKING	 Philosopher 0 is EATING	
Philosopher 2 is waiting for forks	Philosopher 2 picked first fork	Philosopher 1 is waiting for forks	Philosopher 3 is THINKING
Philosopher 1 finished EATING	Philosopher 0 picked second fork	Philosopher 4 is waiting for forks	Philosopher 2 picked second fork
	 Philosopher 0 is EATING	Philosopher 3 finished EATING	 Philosopher 2 is EATING
Philosopher 1 is THINKING	Philosopher 1 is waiting for forks		Philosopher 4 is waiting for forks
Philosopher 2 picked first fork	Philosopher 3 finished EATING	Philosopher 3 is THINKING	Philosopher 1 is waiting for forks
Philosopher 0 picked second fork		Philosopher 2 picked second fork	Philosopher 0 finished EATING
 Philosopher 0 is EATING	Philosopher 3 is THINKING	 Philosopher 2 is EATING	
Philosopher 3 finished EATING	Philosopher 2 picked second fork	Philosopher 0 finished EATING	Philosopher 0 is THINKING
	 Philosopher 2 is EATING		Philosopher 1 picked first fork
Philosopher 3 is THINKING	Philosopher 4 is waiting for forks	Philosopher 0 is THINKING	Philosopher 4 picked first fork
Philosopher 2 picked second fork	Philosopher 0 finished EATING	Philosopher 4 picked first fork	
 Philosopher 2 is EATING		Philosopher 1 picked first fork	
Philosopher 1 is waiting for forks	Philosopher 0 is THINKING	Philosopher 4 picked second fork	
Philosopher 2 finished EATING	Philosopher 4 picked first fork	 Philosopher 4 is EATING	
	Philosopher 1 picked first fork		

No deadlock occurred 